
Comparando algoritmos:

GULOSO X BACKTRACKING

Por Luciano Kohler da Silva e Matheus Eduardo Gomes de Santana

Introdução

O objetivo deste documento consiste em realizar uma análise entre dois paradigmas da computação para resolver um mesmo problema: **Encontrar a quantidade mínima de cores de modo à colorir um grafo sem que quaisquer dois grafos vizinhos tenham a mesma cor.** Para isso, será utilizado um algoritmo **guloso**, e um algoritmo de **backtracking**.

No decorrer deste documento, serão mostrados a **implementação do algoritmo na linguagem C**, junto com uma **análise dos resultados** que foram coletados a partir da execução de ambos os algoritmos a fim de demonstrar a diferença de eficiência temporal de ambos, e comentar um pouco à respeito desta diferença, levantando as **vantagens e desvantagens** de cada implementação.

O grafo utilizado foi dado por meio de uma **matriz de adjacência**, de tamanho **20x20**:

```
0,0,0,0,1,0,1,0,0,1,1,0,1,1,0,1,0,1,1,1
0,0,0,1,1,1,1,0,0,1,0,1,1,1,0,0,1,1,1,1
0,0,0,0,0,0,0,1,1,0,0,1,1,1,1,1,1,1,1,1
0,1,0,0,0,1,0,1,0,1,0,1,1,1,1,1,1,0,1,1
1,1,0,0,0,1,0,0,1,0,1,1,1,1,0,0,1,0,0,1
0,1,0,1,1,0,1,1,0,0,1,0,0,1,1,1,1,1,1,0
1,1,0,0,0,1,0,0,0,1,1,1,1,1,0,1,1,1,0,0
0,0,1,1,0,1,0,0,0,0,1,1,1,1,0,0,1,0,1,1
0,0,1,0,1,0,0,0,0,1,1,0,1,0,0,1,1,0,0,0
1,1,0,1,0,0,1,0,1,0,1,0,1,0,1,1,0,0,1,1
1,0,0,0,1,1,1,1,1,1,0,1,1,0,1,1,1,0,1,1
0,1,1,1,1,0,1,1,0,0,1,0,0,0,1,1,0,1,1,0
1,1,1,1,1,0,1,1,1,1,1,0,0,1,0,1,0,1,1,1
1,1,1,1,1,1,1,1,1,0,0,0,0,1,0,1,0,0,1,0
0,0,1,1,0,1,0,0,0,1,1,1,0,1,0,0,0,1,0,0
1,0,1,1,0,1,1,0,1,1,1,1,1,0,0,0,1,1,1,1
0,1,1,1,1,1,1,1,1,0,1,0,0,0,0,1,0,1,1,1
1,1,1,0,0,1,1,0,0,0,0,1,1,1,1,1,1,0,0,1
1,1,1,1,0,1,0,1,0,1,1,1,1,0,0,1,1,0,0,1
1,1,1,1,1,0,0,1,0,1,1,0,1,1,0,1,1,1,1,0
```

Conteúdo do arquivo "grafo.csv"

Algoritmo Guloso

```
void coloreGuloso(int grafo[N][N]) {
    int resultado[N]; // cor de cada vértice
    int disponivel[N]; // cores que podemos usar
    // (considera o pior caso, onde cada vértice possui uma cor diferente)

    // primeiro vértice tem a "cor 0", e os outros começam não pintados
    resultado[0] = 0;
    for (int i = 0; i < N; i++) resultado[i] = -1;

    // começa com todas as cores disponíveis
    for (int i = 0; i < N; i++) disponivel[i] = 1;

    // começa a colorir os N-1 vértices restantes
    for (int u = 1; u < N; u++) {
        // marca as cores dos vizinhos como indisponíveis
        for (int v = 0; v < N; v++) {
            // se tiver uma aresta para esse vértice, e o vertice
            // possuir alguma cor x, diz que aquela cor x não está disponível
            if (grafo[u][v] && resultado[v] != -1) {
                disponivel[resultado[v]] = 0;
            }
        }

        // procura a primeira cor disponível
        int cor;
        for (cor = 0; cor < N; cor++) if (disponivel[cor]) break;

        resultado[u] = cor; // pinta o vértice com a cor encontrada

        // reinicia a tabela de disponibilidade para a coloração do
        // próximo vértice
        for (int v = 0; v < N; v++) {
            // reverte o que fizemos no primeiro for (colocando as
            // cores dos vizinhos como disponíveis)
            if (grafo[u][v] && resultado[v] != -1) {
                disponivel[resultado[v]] = 1;
            }
        }
    }

    // encontra o número total de cores utilizadas
    int maxCores = 0;
    for (int i = 0; i < N; i++) {
        if (resultado[i] > maxCores) maxCores = resultado[i];
        printf("Vértice %d -> Cor %d\n", i, resultado[i] + 1);
    }
    // maxCores + 1 pois começamos com a "cor 0"
    printf("\nTotal de cores utilizadas: %d\n", maxCores + 1);
}
```

Como o nome sugere, o algoritmo guloso inicia com o uma cor no nodo inicial, e para cada nodo adjacente, ele **tentará colorir com a “menor cor possível”**, caso contrário, ele utilizará outra cor para cumprir a tarefa, repetindo essa estratégia para **todos os nodos**.

Algoritmo Backtracking

```
int coloreGrafo(int grafo[N][N], int cores[], int vertice, int m) {
    // verifica se já passamos por todos os vértices do grafo
    if (vertice == N) {
        return 1;
    }

    // tenta colorir com alguma cor de 1 até 'm'
    // (sendo 'm' o máximo de cores que podemos ter)
    for (int cor = 1; cor <= m; cor++) {
        // verifica se a cor que estamos pode ser
        // utilizada para pintar o vértice que estamos
        if (corLivre(grafo, cores, vertice, cor)) { // Função declarada ao fim dessa função
            cores[vertice] = cor;
            // caso verdadeiro, aplica a recursividade
            if (coloreGrafo(grafo, cores, vertice + 1, m)) {
                return 1;
            }
            // se aplicou a recursividade, e não conseguiu pintar
            // o grafo inteiro, tenta com uma outra cor (cor + 1)
            cores[vertice] = 0;
        }
    }
    // retorna 0 se não foi possível colorir com m cores
    return 0;
}

int corLivre(int grafo[N][N], int cores[], int vertice, int cor) {
    // verifica cada vértice do grafo
    for (int i = 0; i < N; i++) {
        // verifica se o vértice sendo analisado tem adjacência com o vértice que queremos
        // colorir, e, caso possua, se ele tem a mesma cor que queremos colorir o vértice
        if (grafo[vertice][i] && cores[i] == cor) {
            return 0;
        }
    }
    return 1;
}
```

A estratégia por backtracking utiliza um raciocínio mais “inteligente”, **porém custoso**, por meio da recursão, o algoritmo testa se é possível colorir o grafo com n cores, retornando 1 se isso for possível, então, para encontrar a quantidade mínima de cores, basta rodar a **função mais de uma vez**, limitando as cores possíveis em 1, e incrementando esse valor até que seja possível encontrar uma configuração com essas cores.

Resultados e Análise

Após a apresentação dos algoritmos, ambos foram rodados em cima de um mesmo grafo de 20 nodos, os resultados indicaram que **ambos os algoritmos chegaram à um mínimo de 7 cores para colorir todos os nodos**, com o algoritmo de Backtracking tomando aprox. **2 segundos** para executar, e o guloso, apenas **0.5 segundo**.

Em um outro teste com um grafo de 25 nodos, porém, o algoritmo de backtracking **explode instrucionalmente**, isso se dá ao fato de que o algoritmo tem complexidade de tempo $O(k^n)$, sendo **k** a quantidade de cores (que incrementa ao longo da execução), e **n** a quantidade de nodos, se pegarmos de exemplo, **k = 10 e n = 25**, teríamos **10^{25}** , um valor exorbitantemente alto para que o programa consiga terminar em um tempo normal.

Isso se dá pois o algoritmo de coloração de grafos é, como dito anteriormente, um **problema NP-Completo**, ou seja, **mesmo que seja possível que haja uma solução otimizada para o problema, a mesma ainda não foi encontrada**, e por isso, a solução por backtracking só se mostra útil com grafos pequenos. O algoritmo guloso, por outro lado, não possui esse problema pois sua implementação executa em $O(n^2)$, mantenha em mente, porém, que o guloso **NÃO GARANTE** a quantidade mínima de cores para colorir o grafo, apenas apresenta uma solução possível.

Conclusão

- Mesmo que o algoritmo guloso seja idealmente **rápido** para a resolução do problema, **não é possível depender do seu output** como resposta definitiva para o problema, mas sim uma solução possível que pode ou não ser ótima. Se o objetivo é encontrar uma **solução qualquer possível**, a implementação gulosa já é mais do que suficiente, além de mais rápida que uma implementação via backtracking.
- Pela **complexidade do problema** e a **inexistência de algoritmo eficiente** para solução do problema, o uso do algoritmo de backtracking, mesmo que retornando sempre a solução ótima, não pode ser utilizado em grafos grandes ($N \approx 25$) por conta de sua **complexidade de tempo agressiva**. Se o objetivo é encontrar **a melhor solução**, é necessário realizar a implementação com backtracking.
- A comprovação de que uma solução apresentada é válida no contexto do problema é **fácilmente implementável**, pois é só analisar a vizinhança de cada nodo colorido à procura de uma repetição de cores (ou a ausência de uma), essa característica é típica de problemas **NP-Completo**s.